Midterm Exam

- Due No Due Date
- Points None
- Available after Oct 12 at 2pm

Midterm: Reading an Accelerometer, and Proving It

Two weeks. You will bring up a new sensor over I²C, verify that the numbers coming out of it are real, and then characterise the bus itself.

What this midterm is actually testing

You can generate an I²C state machine. You proved that in Lab 6, and you are welcome to use either the machine you wrote in Lab 5 or the one you generated in Lab 6.

What you cannot generate is a board that reads 1 g. Implementation is no longer the hard part of this course. **Verification is.** Most of the marks here are for showing that your numbers are right, not for producing numbers.

Why gravity is the referee

Every sensor you have read so far returned a number you could not independently check. Room temperature looked plausible. So did a slightly wrong room temperature.

An accelerometer is different. Sitting still on the bench, it measures gravity, and gravity has a magnitude you already know. Whatever orientation you put the board in:

```
|a| = sqrt(ax² + ay² + az²)  =  1 g
```

No reference instrument, no calibration, no trusting the datasheet. Rotate the board and the individual axes change completely, but that one number must not. **Physics adjudicates.**

And it catches essentially every realistic way the read can be wrong. Each fault leaves its own distinct signature in a six-orientation table — a swapped byte does not look like a missing sign extension, and neither looks like a register pointer that failed to advance. Reading those signatures is part of Report 1, so resist looking up what they are before you have your own data in front of you.

The one worth knowing about in advance

Multi-byte reads on this part require something in the register address that is not the default, and the datasheet states it plainly. Get it wrong and your code runs, returns plausible-looking signed numbers, and is completely wrong. It is the clearest example in this course of output that is syntactically perfect and silently false.

What Is Due, and When

Item	Points	Due
Midterm – Part 1: the sensor (in pairs, demonstrated)	40	First 60 minutes of the session at the end of week 1
Midterm – Report 1: the sensor (individual)	20	First 60 minutes of the session at the end of week 1
Midterm – Part 2: the bus (in pairs, demonstrated)	25	First 60 minutes of the session at the end of week 2
Midterm – Report 2: the bus (individual)	15	First 60 minutes of the session at the end of week 2
Total	**100**	

Each demonstration and its report are due together, within the first 60 minutes of the session at the end of its week. Report 1 is deliberately due a week after you take the data rather than the same evening, so the writing is analysis and not archaeology. Late submission costs 10% per hour.

The two deadlines are a week apart on purpose

Part 1 is due at the end of **week 1**, not week 2. This is not to compress the work. Part 2 measures the behaviour of the bus that your Part 1 traffic runs on, so the two are genuinely sequential — you need the sensor talking before there is anything to look at with a scope.

It also means that if something is not working, you find out with a week left rather than an evening. Bring whatever you have to the week 1 demonstration even if it is not finished; that session is as much for diagnosis as for marks.

> Each part is a full session's work on its own. Part 1 is bench work at your own station; Part 2 needs the oscilloscope and the marker output you built in Lab 7. Neither compresses well.

Files for This Midterm

Folder	Contents
`midterm/reference`	LSM303DLHC datasheet
`Shared Files/code`	`xem7310_v1_fixed.xdc`, the board constraints, identical in every lab
`Shared Files/reference`	Sensor board layout and schematic, Opal Kelly FrontPanel documentation

No bit file and no reference Python. You bring your own I²C master.

The Sensor

The part is the **LSM303DLHC** (U5 on the schematic). It carries **two independent devices in one package** — a three-axis accelerometer and a three-axis magnetometer — sitting on the same two wires but answering to **different I²C addresses**.

Three things are new compared with the ADT7420:

What is new	Why it matters
Two devices on one bus	You address each separately. Getting one working tells you nothing about the other.
Configuration is required	The accelerometer powers up in a low-power state and returns nothing useful until you write a control register. So does the magnetometer. A read that returns all zeros usually means you skipped this.
Multi-byte reads	Six bytes for three axes. The register pointer has to advance between bytes, and that is not the default behaviour.

Read the datasheet properly before you write anything

Find for yourself: both I²C addresses, the control register that wakes the accelerometer and its documented power-on value, the output register addresses, the full-scale setting and its scale factor, and what has to happen for the register pointer to advance during a burst read.

Two traps worth looking for specifically: the accelerometer and the magnetometer do **not** use the same byte order, and they do **not** report their axes in the same sequence. Both are stated plainly in

the datasheet and both will cost you an evening if you assume.

And one structural thing: on this part **the accelerometer has no identification register**. There is no fixed constant to read back from it. Part 1a is about building a known-answer test anyway.

What You May Use

- Either I²C state machine **you built** — your Lab 5 long-form version or your Lab 6 generated one. Say in Report 1 which you used and why.
- The oscilloscope marker output on PMOD that you added in Lab 7.
- Language models, freely, for both the firmware and the instrument control.

Bring your own I\u00b2C

No bit file and no reference Python this time. You have had a working I²C master since Lab 5 and a second one since Lab 6 — use whichever you prefer, and say in Report 1 which it was.

The protocol itself is not what is being assessed. Pointing a bus you already understand at a new device is a short step from where you are. Almost all of the marks sit further along: on the gravity table, on your board's bus capacitance, and on numbers you measured and can defend.

> Using a model is expected and carries no penalty. Reporting a number you cannot defend does.

What You Upload

Four Canvas items. The code goes to the two demonstrated parts; each report goes to its own.

Assignment	What goes there	When
Midterm – Part 1	`midterm_p1_i2c.v` and `midterm_p1_python.py`	before your week 1 demonstration
Midterm – Report 1	`midterm_report1.pdf`	First 60 minutes of the session at the end of week 1
Midterm – Part 2	`midterm_p2_python.py`	before your week 2 demonstration

Midterm – Report 2	`midterm_report2.pdf`	First 60 minutes of the session at the end of week 2

> The code has to be uploaded before each demonstration, not after.
>
> The TA checks Canvas before starting and will not begin until your files are showing as submitted. A demonstration with no code submitted scores **zero** for that part, whatever the bench did.
>
> This applies separately to each week. Part 1 and Part 2 are graded independently.

Next

Midterm – Part 1 is the sensor. **Part 2** is the bus. **Report 1** and **Report 2** list what to hand in, and each is due with the Part it draws on.